

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 9
Sorting

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

- Introduction and Notation
- Insertion Sort
- Selection Sort
- Shell Sort
- ----------
- *Bucket Sort*
- *Radix Sort*
- *Divide-and-Conquer Sorting*
- *Mergesort*
- *Quicksort*
- *Heapsort*

Introduction

- In computer science and mathematics, a **sorting algorithm** is an algorithm that puts elements of a list in a certain order. The most-used orders are numerical order and lexicographical order.
- Efficient sorting is important to optimizing the use of other algorithms (such as search and merge algorithms) that require sorted lists to work correctly; it is also often useful for producing human-readable output.
- Sorting problem has attracted a great deal of research
  - Bubble sort was analyzed as early as 1956.
  - Useful new sorting algorithms are still being invented . For example, library sort was first published in 2004.

Introduction

- In an external sort, there are so many records to be sorted that they must be kept in external files on disks, tapes, or the like. In an internal sort the records can all be kept internally in high-speed memory. We consider only internal sorting.
- To analyze sorting algorithms, we consider both the number of comparisons of keys and the number of times entries must be moved inside a list, particularly in a contiguous list.
- Both the worst-case and the average performance of a sorting algorithm are of interest. To find the average, we shall consider what would happen if the algorithm were run on all possible orderings of the list (with *n entries, there are n! such* orderings altogether) and take the average of the results.

Introduction

- Computational complexity (worst, average and best behavior) of element comparisons in terms of the size of the list (*n*). For typical sorting algorithms good behavior is *O(n logn)* and bad behavior is $O(n^2)$. Ideal behavior for a sort is *O(n)*.
- Memory usage (and use of other computer resources). In particular, some sorting algorithms are "**in place**". This means that they need only *O(1)* or *O(log n)* memory beyond the items being sorted and they don't need to create auxiliary locations for data to be temporarily stored, as in other sorting algorithms.
- Recursion. Some algorithms are either recursive or non-recursive, while others may be both (e.g., merge sort).



Application of Sorting

- 唯一性测试——如何测试一个给定序列S中的元素是否两两相异?
- 删除序列S中的重复元素
- 中位数(median element)/第k小元素的选择
- 频率统计——哪个元素在序列S中出现的次数最多?
- Find closest pair
- 集合的交与并
- 寻找目标数对——给定一个目标数z,如何检测是否存在两个整数x,y∈ S，使得x+y=z?

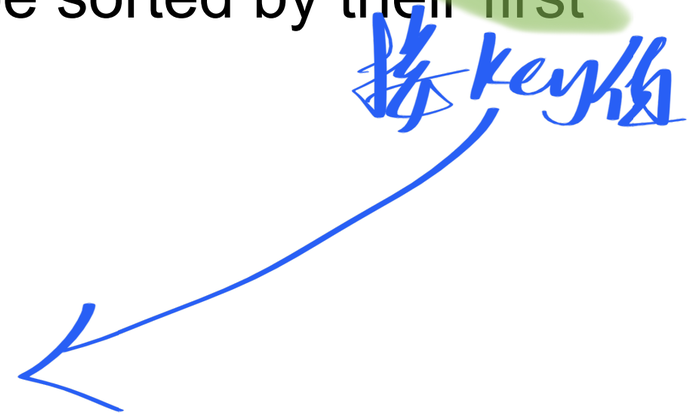
Stability

- Stability: **stable sorting algorithms** maintain the relative order of records with equal keys (i.e., values).
- Assume that the following pairs of numbers are to be sorted by their first component:

  (4, 1) (3, 2) (3, 3) (5, 4)
- Sorted result

  (3, 2) (3, 3) (4, 1) (5, 4) (order maintained)

  (3, 3) (3, 2) (4, 1) (5, 4) (order changed)



Stability

- Unstable sorting algorithms can be specially implemented to be stable.
- One way of doing this is to artificially extend the key comparison, so that comparisons between two objects with otherwise equal keys are decided using the order of the entries in the original data order as a tie-breaker. Remembering this order, however, often involves an additional computational cost.

Sortable Lists

- To write efficient sorting programs, we must access the private data members of the lists being sorted. Therefore, we shall add sorting functions as methods of our basic List data structures. The augmented list structure forms a **new ADT** that we shall call a Sortable List, with the following form:

```
template <class Record>
class Sortable_list: public List<Record> {
public:                    //  Add prototypes for sorting methods here.
private:                   //  Add prototypes for auxiliary functions here.
};
```

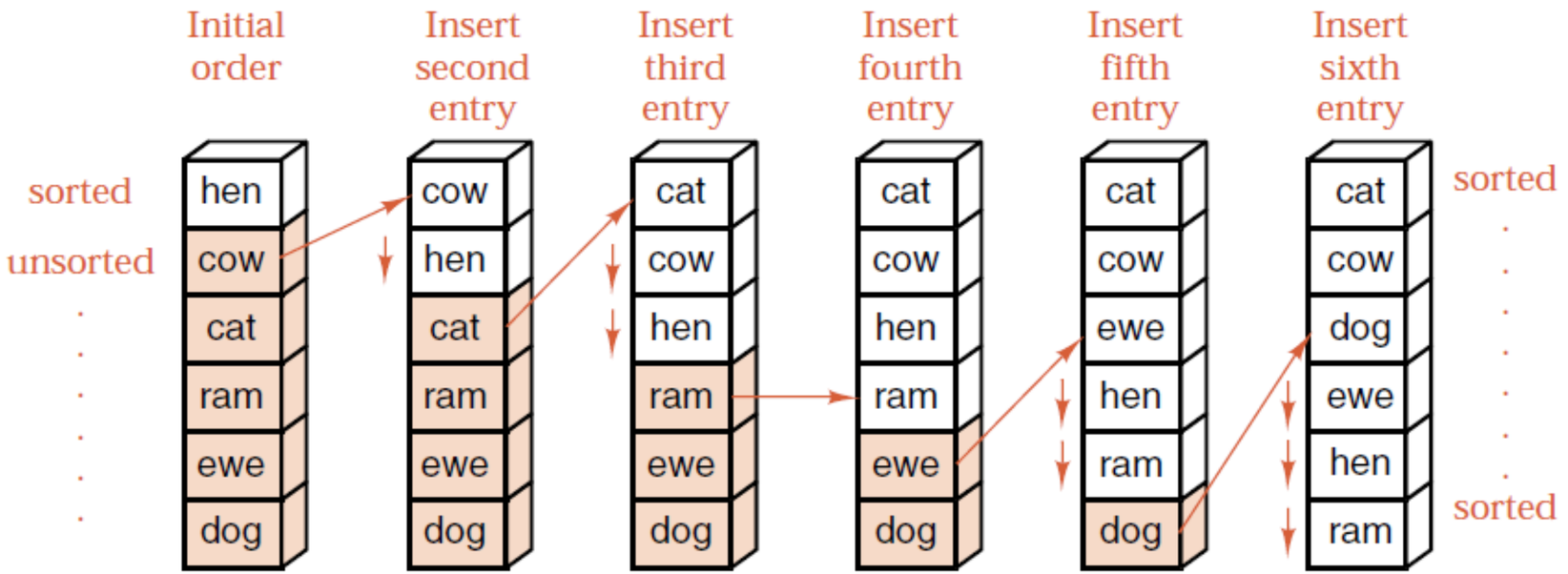
Every Record has an associated key of type Key. A Record can be implicitly converted to the corresponding Key. The keys (hence also the records) can be compared under the operations ‘ < ,’ ‘ > ,’ ‘ >= ,’ ‘ <= ,’ ‘ == ,’ and ‘ != .’

Ordered Insertion

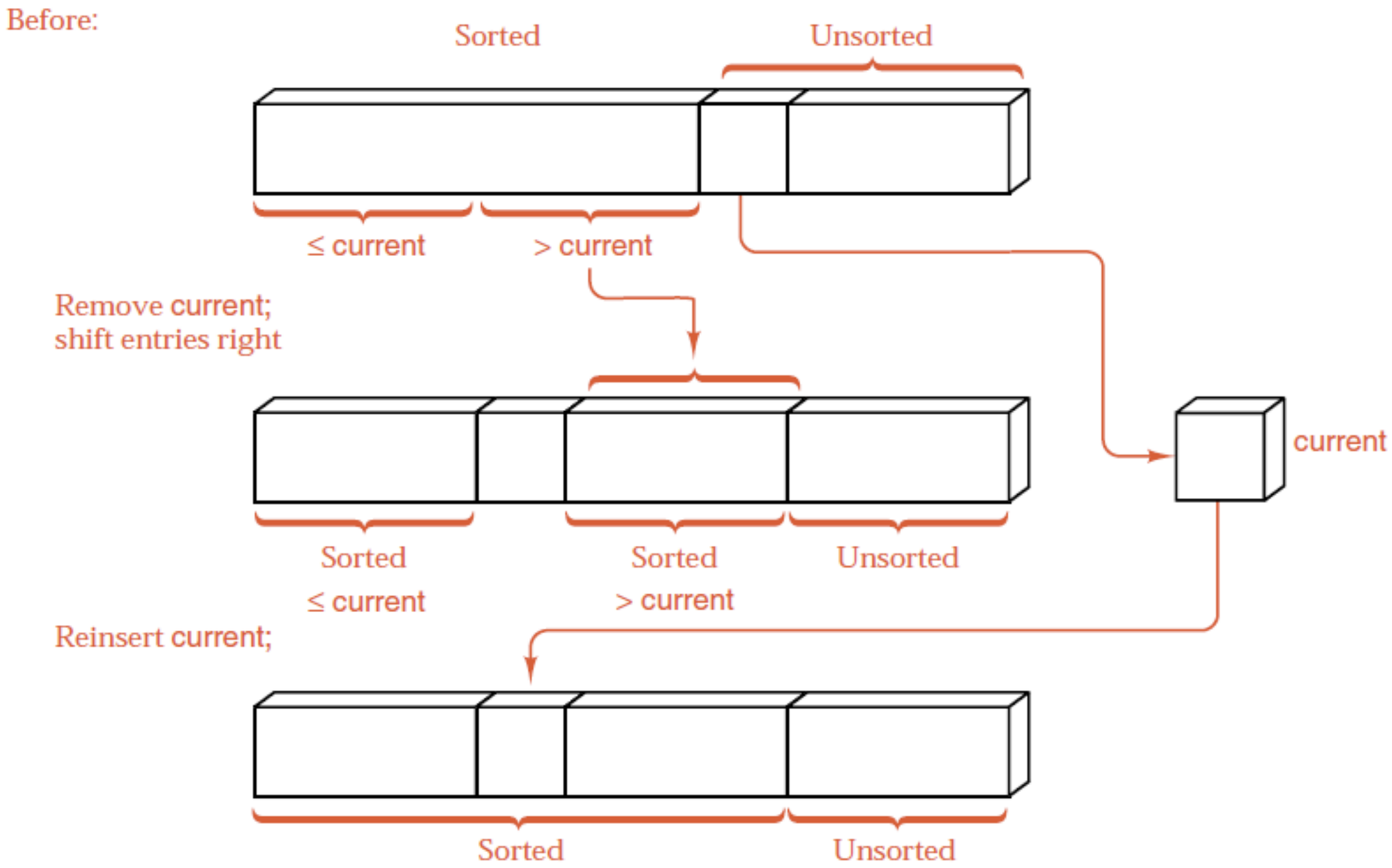
- An *ordered list* is an abstract data type, defined as a list in which each entry has a key, and such that the keys are in order.
- For ordered lists, we shall often use two new operations.
  - One operation *retrieves* an entry with a specified key from the ordered list. Retrieval by key from an ordered list is exactly the same as searching.
  - The second operation, *ordered insertion*, inserts a new entry into an ordered list by using the key in the new entry to determine where in the list to insert it.
- Note that ordered insertion is not uniquely specified if the list already contains an entry with the same key as the new entry, since the new entry could go into more than one position.

Insertion Sort

- Basic Idea：Insertion sort for a list is based on the idea of insertion into an ordered list. To sort an unordered list, we think of removing its entries one at a time and then inserting each of them into an initially empty new list, always keeping the entries in the new list in the proper order according to their keys.



Insertion Sort



Insertion Sort Example

Insertion Sort Example							
Original	*34*	*8*	*64*	*51*	*32*	*21*	**Moved**
After p=1	*8*	*34*	*64*	*51*	*32*	*21*	***1***
After p=2	*8*	*34*	*64*	*51*	*32*	*21*	***0***
After p=3	*8*	*34*	*51*	*64*	*32*	*21*	***1***
After p=4	*8*	*32*	*34*	*51*	*64*	*21*	***3***
After p=5	*8*	*21*	*32*	*34*	*51*	*64*	***4***

Insert Sort

- A simple implementation:

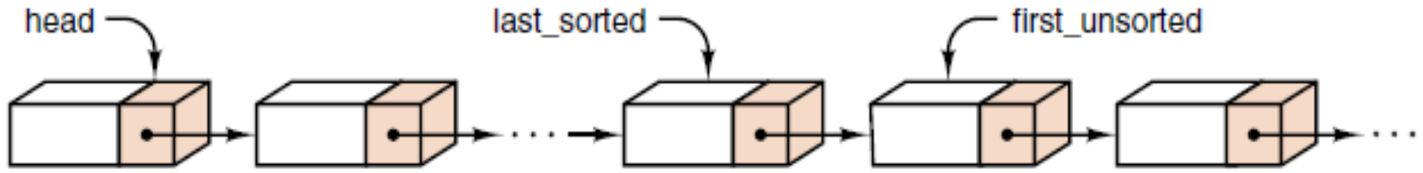
```
for (i = 1; i < n; i++) {
        key = a[i];
        for (j = i - 1; j >= 0 && a[j] > key; j--)
                a[j + 1] = a[j];
        a[j + 1] = key;
}
```

Linked Insertion Sort

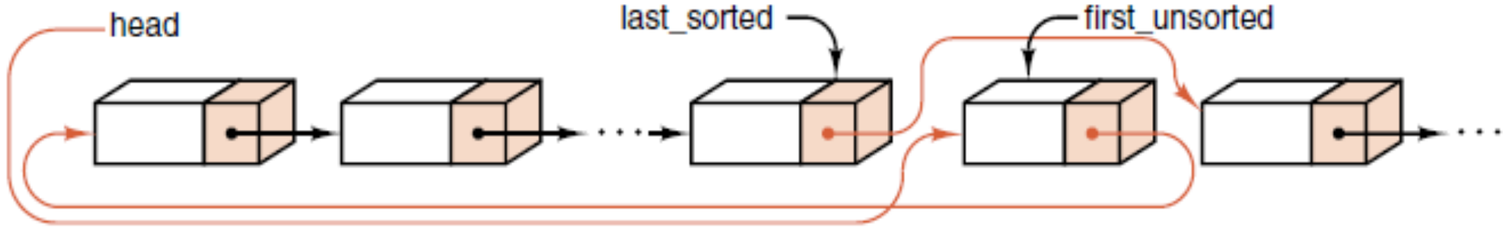
- Traverse the original list, taking one entry at a time and inserting it in the proper position in the sorted list.
- A ***sentinel*** is an extra entry added to one end of a list to ensure that a loop will terminate without having to include a separate check. Since last_sorted->next == first_unsorted, the node *first_unsorted is already in position to serve as a sentinel for the search, and the loop moving **current** is simplified.
- A list with 0 or 1 entry is already sorted, so by checking these cases separately we avoid trivialities elsewhere.
- The contiguous version searches the sorted sublist in reverse order, while the linked version searches it in increasing order of position within the list.

Linked Insertion Sort

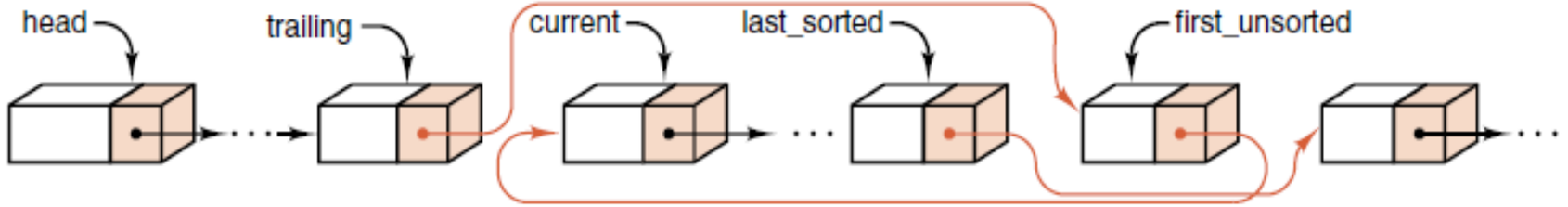
Partially sorted:



Case 1: *first_unsorted belongs at head of list



Case 2: *first_unsorted belongs between *trailing and *current



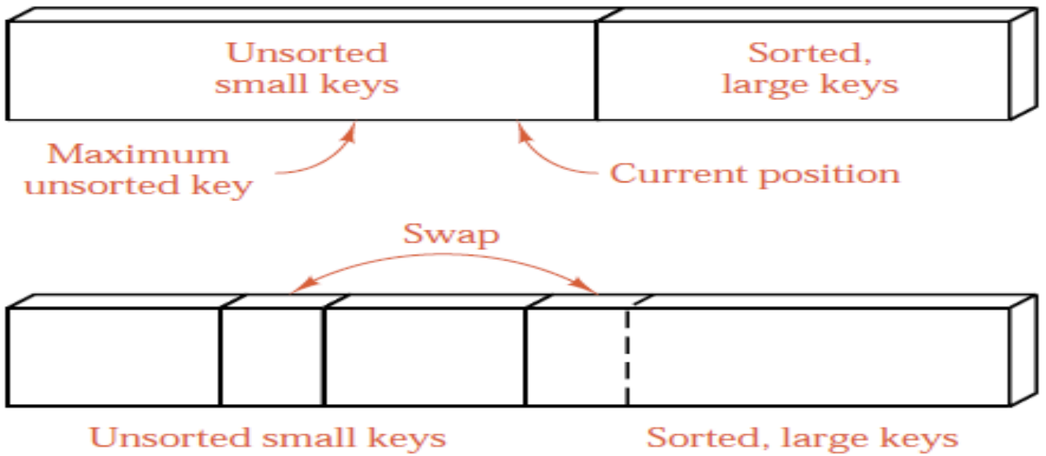
Analysis of Insertion Sort

- The average number of comparisons for insertion sort applied to a list of *n items in random order is* $1/4n^2+O(n)$.
- The average number of assignments of items for insertion sort applied to a list of *n items in random order is also* $1/4n^2+O(n)$.
- The best case for contiguous insertion sort occurs when the list is already in order, when insertion sort does nothing except $n - 1$ comparisons of keys.
- The worst case for contiguous insertion sort occurs when the list is in reverse order.
- Insertion sort is stable, it does not change the relative order of elements with equal keys.
- It is In-place, i.e., only requires a constant amount O(1) of additional memory space.

Selection Sort

- Insertion sort has one disadvantage: insertion of an entry may require many entries to be moved. This can be expensive especially when the entries are large.
- Basic Idea of Selection Sort
  1. Scan the unsorted list and find the largest, then exchange this entry with the entry in the last position.
  2. Repeat the process on the unsorted list until the length of unsorted list is 1.

General step:



Selection Sort

- A simple implementation:

```
for (i = 0; i < n; i++) {
   k = i;
   for (j = i + 1; j < n; j++)
      if (a[j] < a[k]) k = j;
   swap(a[i], a[k]);
}
```

Selection Sort

- Selection sort minimizes data movement, it is useful for contiguous lists with large entries for which movement of entries is expensive.

选择排序

Original： 21，25，49，25*，16，08

p=1 08，25，49，25*，16，21

p=2 08，16，49，25*，25，21

p=3 08，16，21，25*，25，49

p=4 08，16，21，25*，25，49

p=5 08，16，21，25*，25，49

时间效率：$O(n^2)$ ——虽移动次数较少，但比较次数仍多。

空间效率：O(1) ——因为仅占用1个缓冲单元！

算法的稳定性：不稳定——因为排序时，25*到了25的前面。

Analysis of Selection Sort

- The number of comparisons:

  (n-1)+(n-2)+…+1 = ½ n(n+1) ≈ ½ n² + O(n)
- The number of assignments: 3(n-1)

- Analysis and comparisons:

	Selection	*Insertion (average)*
Assignments of entries	$3.0n + O(1)$	$0.25n^2 + O(n)$
Comparisons of keys	$0.5n^2 + O(n)$	$0.25n^2 + O(n)$

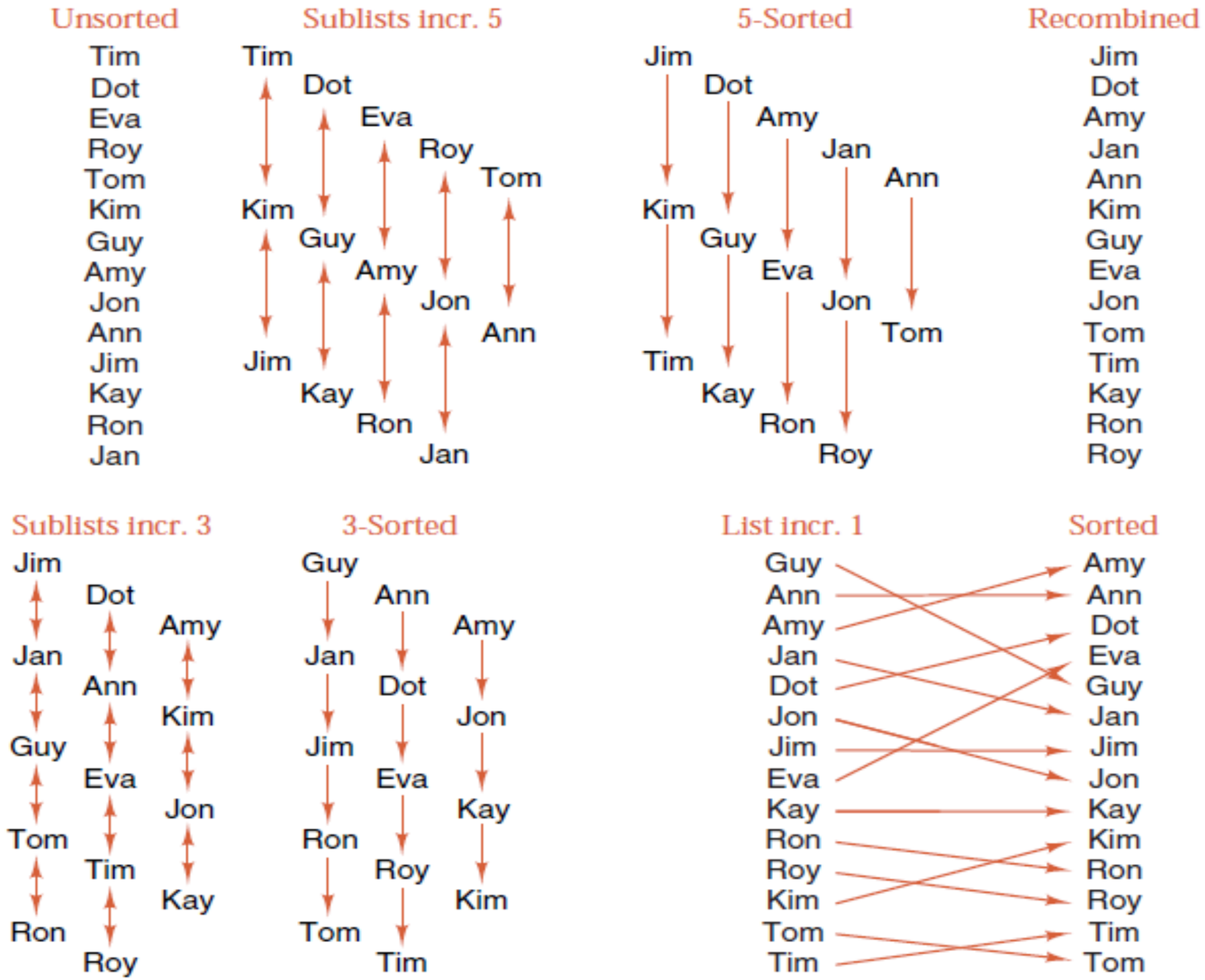
Shell Sort

- *Shell sort* was invented by *Donald Shell* in 1959. It improves upon bubble sort and insertion sort by moving out of order elements more than one position at a time.
- Insertion sort is inefficient in moving entries only one position at a time. It takes several insertion for a entry in its correct position. The reason is it only compares adjacent entries.



- How to reduce the number of movements?
- Idea: If we compare entries far apart, then it could sort the entries far apart. Finally, the increment between keys being compared is reduced to 1.

Shell Sort


Unsorted
Tim
Dot
Eva
Roy
Tom
Kim
Guy
Amy
Jon
Ann
Jim
Kay
Ron
Jan
Sublists incr. 5
Tim
Dot
Eva
Roy
Tom
Kim
Guy
Amy
Jon
Ann
Jim
Kay
Ron
Jan
5-Sorted
Jim
Dot
Amy
Jan
Ann
Kim
Guy
Eva
Jon
Tom
Tim
Kay
Ron
Roy
Recombined
Jim
Dot
Amy
Jan
Ann
Kim
Guy
Eva
Jon
Tom
Tim
Kay
Ron
Roy
Sublists incr. 3
Jim
Dot
Amy
Jan
Ann
Kim
Guy
Eva
Jon
Tom
Tim
Kay
Ron
Roy
3-Sorted
Guy
Ann
Amy
Jan
Dot
Jon
Jim
Eva
Kay
Ron
Roy
Kim
Tom
Tim
List incr. 1
Sorted
Guy
Ann
Amy
Jan
Dot
Jon
Jim
Eva
Kay
Ron
Roy
Kim
Tom
Tim
Amy
Ann
Dot
Eva
Guy
Jan
Jim
Jon
Kay
Kim
Ron
Roy
Tim
Tom

Contiguous implementation

```
template <class Record>
void Sortable_list<Record> :: shell_sort( )
/* Post:  The entries of the Sortable_list have been rearranged so that the keys in all
          the entries are sorted into nondecreasing order.
   Uses: sort_interval  */


{
   int increment,                    //   spacing of entries in sublist
       start;                        //   starting point of sublist
   increment = count;


   do {
      increment = increment/3 + 1;
      for (start = 0;  start < increment;  start++)
         sort_interval(start, increment);   //   modified insertion sort
   } while (increment > 1);
}
```

Shell Sort Example

例：关键字序列 T=(49,38,65,97,76,13,27,49*,55,04），
请写出希尔排序的具体实现过程。

r[i]	0	1	2	3	4	5	6	7	8	9	10
初态：		49	38	65	97	76	13	27	49*	55	04
第1趟 (inc=5)		13	27	49*	55	04	49	38	65	97	76
第2趟 (inc=3)		13	04	49*	38	27	49	55	65	97	76
第3趟 (inc=1)		04	13	27	38	49*	49	55	65	76	97

Shell Sort Example

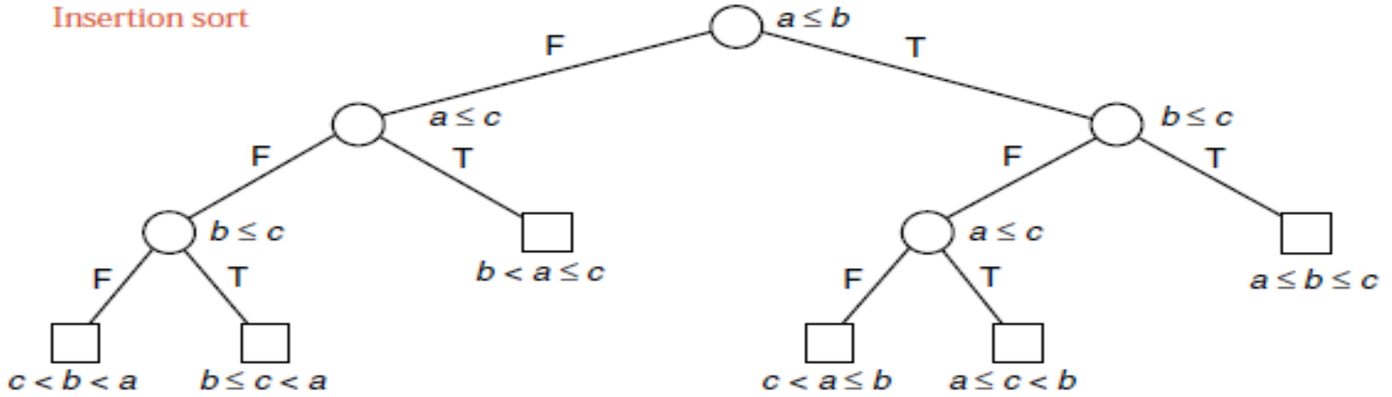
算法分析：开始时*inc* 的值较大，子序列中的对象较少，排序速度较快；随着排序进展，*inc* 值逐渐变小，子序列中对象个数逐渐变多，由于前面工作的基础，大多数对象已基本有序，所以排序速度仍然很快。

时间效率：$O(n^{4/3})$ ～$O(n^2)$——希尔增量、Hibbard增量等
空间效率：O(n)
算法的稳定性：不稳定——因为49*排序后却到了49的前面

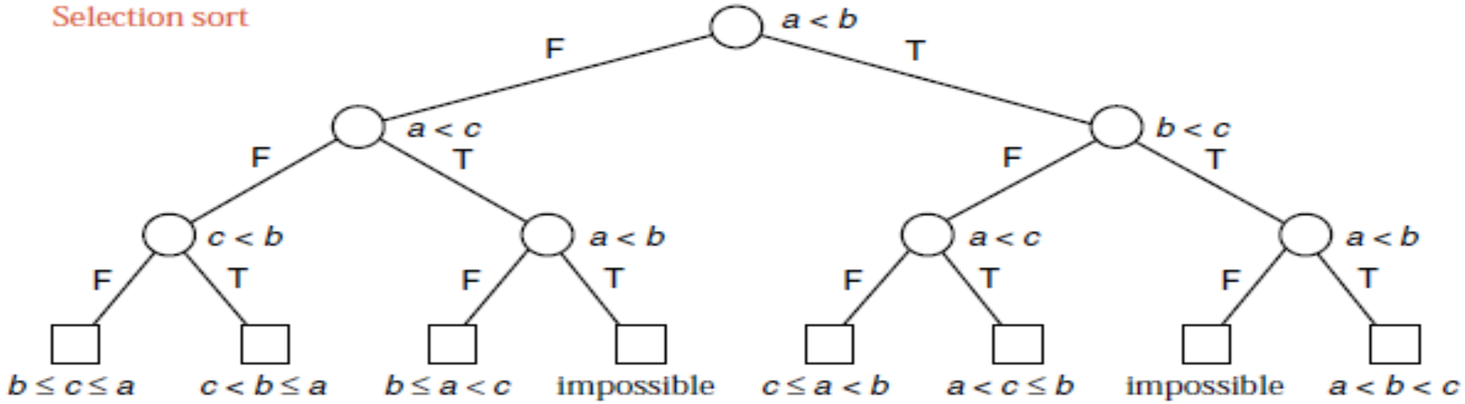
- Exercise:
  - Use Shell Sort to sort [256，301，751，129，937，863，742，694，076，438]
  - increment = 5, 3, 1

Lower Bounds for Sorting

Insertion sort



Selection sort



THEOREM 8.2 Any algorithm that sorts a list of n entries by use of key comparisons must, in its worst case, perform at least $\lceil \lg n! \rceil$ comparisons of keys, and, in the average case, it must perform at least $\lg n!$ comparisons of keys.

Approximation for factorials

- Stirling's approximation

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1+\Theta\left(\frac{1}{n}\right)\right)$$

$$n! = o(n^n)$$

$$n! = \omega(2^n)$$

$$\log(n!) = \Theta(n \log n)$$

Thank you!

